Lab 8 - Digital Filter Design (part 2)

8.1 Introduction

This is the second part of a three week laboratory in digital filter design. The two weeks of the laboratory covered some basic examples of FIR and IIR filters, and then introduced the concepts of filter design. In this week we will cover more systematic methods of designing FIR filters.

8.2 Filter Design Using Standard Windows

Download **DTFT.m** for the following section.

We can generalize the idea of truncation by using different windowing functions to truncate an ideal filter's impulse response. Note that by simply truncating the ideal filter's impulse response, we are actually multiplying (or "windowing") the impulse response by a shifted $rect(\ )$ function. This particular type of window is called a **rectangular** window. In general, the impulse response $h[n]$ of the designed filter is related to the impulse response $h_{ideal}[n]$ of the ideal filter by the relation

$$h[n] = w[n]h_{ideal}[n] \tag{8.1}$$

where $w[n]$ is an N-point window. We assume that

$$h_{ideal}[n] = \frac{\omega_c}{\pi}\operatorname{sinc}\left(\frac{\omega_c}{\pi}\left(n - \frac{N-1}{2}\right)\right) \tag{8.2}$$

where ω_c is the cutoff frequency and N is the desired window length.

The rectangular window is defined as

$$w[n] = \begin{cases} 1 & n = 0,1,\dots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{8.3}$$

As we have seen in the last lab, the DTFT of $w[n]$ for $N = 21$ is shown in Figure 7.7. The rectangular window is usually not preferred because it leads to the large stopband and passband ripple as shown in Figure 7.6.

More desirable frequency characteristics can be obtained by making a better selection for the window, $w[n]$. In fact, a variety of raised cosine windows are widely used for this purpose. Some popular windows are listed below.

1. Hanning window (as defined in Matlab, command hann(N)):

$$w[n] = \begin{cases} 0.5 - 0.5\cos\dfrac{2\pi n}{N-1} & n = 0,1,\dots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{8.4}$$

2. Hamming window

$$\omega[n] = \begin{cases} 0.54 - 0.46\cos\frac{2\pi n}{N-1} & n = 0,1,\dots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{8.5}$$

3. Blackman window

$$\omega[n] = \begin{cases} 0.42 - 0.5\cos\frac{2\pi n}{N-1} + 0.08\cos\frac{4\pi n}{N-1} & n = 0,1,\dots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{8.6}$$

In filter design using different truncation windows, there are two key frequency domain effects that are important to the design: the transition band **roll-off**, and the passband and stopband **ripple** (see Figure 7.4 in last section). There are two corresponding parameters in the spectrum of each type of window that influence these filter parameters. The filter's roll-off is related to the **width** of center lobe of the window's magnitude spectrum. The **ripple** is influenced by the ratio of the mainlobe amplitude to the first sidelobe amplitude (or difference if using a dB scale). These two window spectrum parameters are **not** independent, and you should see a trend as you examine the spectra for different windows. The theoretical values for the **mainlobe width** and the **peak-to-sidelobe amplitude** are shown in Table 8.1.

Table 8.1: Approximate spectral parameters of truncation windows. See reference [1].

Window (length N)	Mainlobe width	Peak-to-sidelobe amplitude (dB)
Rectangular	$4\pi/N$	$-13dB$
Hanning	$8\pi/N$	$-32dB$
Hamming	$8\pi/N$	$-43dB$
Blackman	$12\pi/N$	$-58dB$

Plot the rectangular, Hamming, Hanning, and Blackman window functions of length 21 on a single figure using the subplot command. You may use the Matlab commands **hamming**, **hann**, and **blackman**. Then compute and plot the DTFT magnitude of each of the four windows. Plot the magnitudes on a decibel scale (i.e., plot $20\log_{10}\left|W(e^{j\omega})\right|$). Download and use the function **DTFT.m** to compute the DTFT.

<u>NOTE: Use at least 512 sample points in computing the DTFT by typing the command DTFT(window,512).</u>

Measure the null-to-null mainlobe width (in rad/sample) and the peak-to-sidelobe amplitude (in dB) from the logarithmic magnitude response plot for each window type. The Matlab command **zoom** is helpful for this. Make a table with these values and the theoretical ones.

Now use a Hamming window to design a lowpass filter $h[n]$ with a cutoff frequency of $\omega_c = 2.0$ and length 21. Note: You need to use (8.1) and (8.2) for this design. In the same figure, use subplots to plot the filter's impulse response, and the magnitude of the filter's DTFT in decibels.

INLAB REPORT:

1. Submit the figure containing the time domain plots of the four windows.
2. Submit the figure containing the DTFT (in decibels) of the four windows.
3. Submit the table of the measured and theoretical window spectrum parameters. Comment on how close the experimental results matched the ideal values. Also comment on the relation between the width of the mainlobe and the peak-to-sidelobe amplitude.
4. Submit the plots of your designed filter's impulse response and the magnitude of the filter's DTFT.

8.3 Filter Design Using the Kaiser Window

Download **nspeech2.mat** for the following section.

The standard windows of the "Filter Design Using Standard Windows" section are an improvement over simple truncation, but these windows still do not allow for arbitrary choices of transition bandwidth and ripple. In 1964, James Kaiser derived a family of near-optimal windows that can be used to design filters which meet or exceed any filter specification. The Kaiser window depends on two parameters: the window length N, and a parameter β which controls the shape of the window. Large values of β reduce the window side lobes and therefore result in reduced passband and stopband ripple. The only restriction in the Kaiser filter design method is that the passband and stopband ripple must be equal in magnitude. Therefore, the Kaiser filter must be designed to meet the smaller of the two ripple constraints:

$$\delta = \min\{\delta_p, \delta_s\} \tag{8.7}$$

The Kaiser window function of length N is given by

$$\omega[n] = \begin{cases} \dfrac{I_0\left(\beta\sqrt{\dfrac{n(N-1-n)}{N-1}}\right)}{I_0(\beta)} & n = 0,1,\dots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{8.8}$$

Where $I_0(\cdot)$ is the zero-th order modified Bessel function of the first kind, N is the length of the window, and β is the shape parameter.

Kaiser found that values of β and N could be chosen to meet any set of design parameters, $(\delta, \omega_P, \omega_S)$, by defining $A = -20\log_{10}\delta$ and using the following two equations:

$$\beta = \begin{cases} 0.1102(A-8.7) & A > 50 \\ \{0.5842(A-21)^{0.4} + 0.07886(A-21) & 21 \le A \le 50 \\ 0.0 & A < 21 \end{cases} \tag{8.9}$$

$$N = \left\lceil 1 + \frac{A-8}{2.285(\omega_s - \omega_p)} \right\rceil \tag{8.10}$$

where $\lceil \cdot \rceil$ is the **ceiling** function, i.e. $\lceil x \rceil$ is the smallest integer which is greater than or equal to x.

To further investigate the Kaiser window, plot the Kaiser windows and their DTFT magnitudes (in dB) for $N = 21$ and the following values of β:

- $\beta = 0$
- $\beta = 1$
- $\beta = 5$

For each case use at least 512 points in the plot of the DTFT.

HINT: To create the Kaiser windows, use the Matlab command kaiser(N,beta) command where N is the length of the filter and beta is the shape parameter β. To insure at least 512 points in the plot use the command DTFT(window,512) when computing the DTFT.

INLAB REPORT: Submit the plots of the 3 Kaiser windows and the magnitude of their DTFT's in decibels. Comment on how the value β affects the shape of the window and the sidelobes of the DTFT.

Next use a Kaiser window to design a lowpass filter, $h(n)$, using equations (8.1) and (8.2), to remove the noise from the signal in **nspeech2.mat**. To do this, use equations (8.9) and (8.10) to compute the values of β and N that will yield the following design specifications:

$$\omega_p = 1.8,\ \omega_c = 2.0,\ \omega_s = 2.2,\ \delta_p = 0.05,\ \delta_s = 0.005$$

The lowpass filter designed with the Kaiser method will automatically have a cut-off frequency centered between ω_p and ω_s.

$$\omega_c = \frac{\omega_p + \omega_s}{2}$$

Plot the magnitude of the DTFT of $h[n]$ for $|\omega| < \pi$. Create three plots in the same figure: one that shows the entire frequency response, and ones that zoom in on the passband and stopband ripple, respectively. Mark ω_p, ω_s, δ_p, and δ_s on these plots where appropriate. Note: Since the ripple is measured on a magnitude scale, DO NOT use a decibel scale on this set of plots.

From the Matlab prompt, compute the stopband and passband ripple (do not do this graphically). Record the stopband and passband ripple to three decimal places.

HINT: To compute the passband ripple, find the value of the DTFT at frequencies corresponding to the passband using the command H(abs(w)<=1.8) where H is the DTFT of $h[n]$ and w is the corresponding vector of frequencies. Then use this vector to compute the passband ripple. Use a similar procedure for the stopband ripple.

Filter the noisy speech signal in **nspeech2.mat** using the filter you have designed. Then compute the DTFT of 400 samples of the filtered signal starting at time $n = 20000$ (i.e. 20001:20400). Plot the magnitude of the DTFT samples in decibels versus frequency in radians for $|\omega| < \pi$. Compare this with the spectrum of the noisy speech signal shown in Figure 7.5. Play the noisy and filtered speech signals back using **sound** and listen to them carefully.

INLAB REPORT: Do the following:

1. Submit the values of β and N that you computed.

2. Submit the three plots of the filter's magnitude response. Make sure the plots are labeled.
3. Submit the values of the passband and stopband ripple. Does this filter meet the design specifications?
4. Submit the magnitude plot of the DTFT in dB for the filtered signal. Compare this plot to the plot of Figure 7.5.
5. Comment on how the frequency content and the audio quality of the filtered signal have changed after filtering.

8.4 FIR Filter Design Using Parks-McClellan Algorithm

Click nspeech2.mat for help on the **firpm** function for Parks-McClellan filter design. Download the data file nspeech2.mat for the following section.

Kaiser windows are versatile since they allow the design of arbitrary filters which meet specific design constraints. However, filters designed with Kaiser windows still have a number of disadvantages. For example,

- Kaiser filters are not guaranteed to be the minimum length filter which meets the design constraints.
- Kaiser filters do not allow passband and stopband ripple to be varied independently.

Minimizing filter length is important because in many applications the length of the filter determines the amount of computation. For example, an FIR filter of length N may be directly implemented in the time domain by evaluating the expression

$$y(n) = \sum_{k=0}^{N-1} x(n-k)h(k) \tag{8.11}$$

For each output value $y(n)$ this expression requires N multiplies and $N-1$ additions.

Oftentimes $h(n)$ is a symmetric filter so that $h(n) = h(N-1-n)$. If the filter $h(n)$ is symmetric and N is even, then (6.16) may be more efficiently computed as

$$y(n) = \sum_{k=0}^{N/2-1} \big(x(n-k) + x(n-N+1+k)\big)h(k) \tag{8.12}$$

This strategy reduces the computation to $N/2$ multiplies and $N-1$ adds for any value of N. Note that the computational effort is linearly proportional to the length of the filter.

The Kaiser filters do not guarantee the minimum possible filter length. Since the filter has equal passband and stopband ripple, it will usually exceed design requirements in one of the two bands; this results in an unnecessarily long filter. A better design would allow the stopband and passband constraints to be specified
separately.

In 1972, Parks and McClellan devised a methodology for designing symmetric filters that minimize filter length for a particular set of design constraints $\{\omega_p, \omega_s, \delta_p, \delta_s\}$. The resulting filters minimize the maximum error between the desired frequency response and the actual frequency response by spreading the approximation error uniformly over each band. The Parks and McClellan

algorithm makes use of the Remez exchange algorithm and Chebyshev approximation theory. Such filters that exhibit **equiripple** behavior in both the passband and the stopband, and are sometimes called equiripple filters.

As with Kaiser filters, designing a filter with the Parks and McClellan algorithm is a two-step process. First the length (i.e. order) of the filter must be computed based on the design constraints. Then the optimal filter for a specified length can be determined. As with Kaiser windows, the filter length computation is approximate so the resulting filter may exceed or violate the design constraints. This is generally not a
problem since the filter can be redesigned for different lengths until the constraints are just met.

The Matlab command for computing the approximate filter length is

[n, fo, mo, w] = firpmord(f, m, ripple, 2*pi) where the inputs are:

- **f** - vector containing an even number of band edge frequencies. For a simple low pass filter, **f = [wp,ws]** ,where **wp** and **ws** are the passband and stopband frequencies, respectively.
- **m** - vector containing the ideal filter magnitudes of the filter in each band. For a simple low pass filter **m = [1, 0]** .
- **ripple** - vector containing the allowed ripple in each band. For a simple low pass filter **ripple = [delta_p, delta_s]**, where **delta_p** and **delta_s** are the passband and stopband ripples, respectively.
- **2*pi** - value, in radians, that corresponds to the sampling frequency.

The outputs of the command are **n = filter length - 1**, and the vectors **fo**, **mo**, and **w** which are intermediate filter parameters.

Once the filter length, **n**, is obtained, the Matlab command for designing a Parks-McClellan filter is **b = firpm(n, fo, mo, w)** . The inputs **n**, **fo**, **mo**, and **w** are the corresponding outputs of **firpmord**, and the output **b** is a vector of FIR filter coefficients such that

$$H(z) = b(1) + b(2)z^{-1} + \cdots + b(n+1)z^{-n} \tag{8.13}$$

(What is the impulse response of this filter?)

For further information, read the help document on using Matlab to implement the Parks-McClellan algorithm.

Now design a symmetric FIR filter using **firpmord** and **firpm** in Matlab to meet the design specifications given in the "Filter Design Using the Kaiser Window" (Section 7.3: Filter Design Using the Kaiser Window) section. Compute the DTFT of the filter's response for at least 512 points, and use this result to compute the passband and stopband ripple of the filter that was designed. Adjust the filter length until the minimum order which meets the design constraints is found. Plot the magnitude of the DTFT in dB of the final filter design.

INLAB REPORT

1. Submit the final measured values of filter length, passband ripple, and stopband ripple. How accurate was the filter order computation using Matlab's firpmord? How does the length of this filter compare to the filter designed using a Kaiser window?
2. Submit the plot of the filter's DTFT. How does the frequency response of the Parks-McClellan filter compare to the filter designed using the Kaiser window? Comment on the shape of both the passband and stopband.

Use the filter you have designed to remove the noise from the signal nspeech2.mat . Play the noisy and filtered speech signals back using sound and listen to them carefully. Compute the DTFT of 400 samples of the filtered signal starting at time $n = 20{,}001$ (i.e. 20001:20400). Plot the magnitude of the DTFT in decibels versus frequency in radians for $|\omega| < \pi$. Compare this with the spectrum of the noisy speech signal shown in Figure 7.5, and also with the magnitude of the DTFT of the Kaiser filtered signal.

INLAB REPORT: Submit the plot of the DTFT magnitude for the filtered signal. Comment on how the audio quality of the signal changes after filtering. Also comment on any differences in audio quality between the Parks-McClellan filtered speech and the Kaiser filtered speech.